

Comparative Big Data Analysis of the 2024 Flight Delay Dataset

Spark Core, Spark SQL and Hive

Juan Miguel Pinos

Big Data Course – Roma Tre University

June 2026

Abstract

This report presents a comparative Big Data analysis of the 2024 Flight Delay Dataset using Spark Core, Spark SQL and Hive. The dataset contains more than seven million flight records and includes information about airlines, airports, delays, cancellations and delay causes. The objective of the project was not only to compute the requested analytical outputs, but also to compare how different Big Data technologies behave when solving the same aggregation-heavy workload.

Two analyses were implemented. The first one computes airline statistics by departure airport, including the number of flights, arrival-delay statistics, cancellation rate and operating months. The second one builds monthly delay reports by departure airport, separating flights into low, medium and high departure-delay ranges and identifying the most frequent delay or cancellation causes. Before executing the analyses, the original CSV dataset was cleaned, reduced to the relevant attributes and converted into Parquet format.

The experiments were executed both on a local machine and on a small AWS EMR cluster. Spark Core and Spark SQL were evaluated using different input sizes, while Hive was evaluated on the full prepared dataset. The results show that Spark SQL provided the best balance between performance and implementation effort. Spark Core gave more control over the aggregation logic, but required more manual code and produced higher execution times. Hive offered a useful SQL warehouse-style alternative, although it required careful schema alignment when reading the Parquet files generated by Spark.

1 Introduction

Flight delay data are a useful case study for Big Data processing because they combine a large number of records with temporal, categorical and numerical attributes. A single year of flights contains millions of observations and allows several types of analysis: airline performance, airport behaviour, delay severity and cancellation causes. This makes the dataset appropriate not only for computing statistics, but also for comparing alternative processing technologies under the same workload.

The goal of this project was to analyse the 2024 Flight Delay Dataset using three technologies: Spark Core, Spark SQL and Hive. Each technology represents a different way of expressing distributed computation. Spark Core exposes a lower-level RDD-based programming model, Spark SQL provides a higher-level declarative interface over structured data, and Hive follows a warehouse-oriented approach based on external tables and HQL queries.

The original dataset was provided as a CSV file. Since the same data had to be processed several times and with different technologies, directly reading the raw CSV file for every experiment would have introduced unnecessary parsing overhead. For this reason, I implemented a

data preparation phase that selected the relevant columns, normalized delay-related values and stored the result in Parquet format. Parquet was chosen because it is columnar, compact and better suited to analytical workloads than raw CSV.

The project also included a practical infrastructure component. The jobs were executed first on a local machine and then on an AWS EMR cluster using Amazon S3 as storage. This made it possible to compare single-machine execution with a distributed environment. An important lesson from the experiments was that a cluster is not automatically faster: for this dataset size and this small EMR configuration, the overhead of distributed execution was sometimes higher than the benefit of parallelism.

During development, one of the main technical issues appeared when reading the Spark-generated Parquet dataset from Hive. The Hive external table schema had to match the physical schema stored in Parquet. In particular, the field `fl_date` produced schema inspection errors when its Hive definition did not match the type written by Spark. This required adjusting the Hive table definition before the cluster queries could run correctly. This issue was useful because it showed that Big Data pipelines depend not only on the query logic, but also on storage format compatibility and schema interpretation across tools.

The rest of the report is organized as follows. Section 2 describes the dataset and the selected attributes. Section 3 explains the preparation phase. Section 4 presents the implemented analyses. Section 5 discusses the implementation of Spark Core, Spark SQL and Hive, including selected code fragments. Section 6 describes the local and cluster environments. Section 7 presents the results and execution-time comparison. Section 8 discusses the main technical findings. Finally, Section 9 explains how to reproduce the project and Section 10 concludes the report.

2 Dataset Description

The dataset used in this project is the 2024 Flight Delay Dataset. The original file was provided in CSV format and contains flight-level records for commercial flights operated during 2024. Each record describes one flight and includes operational attributes such as airline, origin airport, destination airport, departure delay, arrival delay, cancellation flag and delay-cause information.

After the preparation phase, the dataset used in the experiments contained 7,079,081 records. The original dataset was not included in the GitHub repository because of its size. Instead, the repository contains the code and instructions needed to reproduce the pipeline after placing the CSV file in the expected input directory.

The analyses implemented in this project did not require all the columns of the original dataset. For this reason, only the attributes needed for the selected jobs were kept. Table 1 summarizes the main fields used.

Table 1: Main attributes used in the analyses

Attribute	Purpose
<code>fl_date</code>	Flight date, used to derive the operating month.
<code>op_unique_carrier</code>	Airline carrier code, used in the airline statistics.
<code>origin</code>	Departure airport, used as a grouping key in both analyses.
<code>dest</code>	Destination airport, used to identify routes when needed.
<code>dep_delay</code>	Departure delay in minutes, used to define delay ranges.
<code>arr_delay</code>	Arrival delay in minutes, used to compute delay statistics.
<code>cancelled</code>	Cancellation flag, used to compute cancellation rates.
<code>cancellation_code</code>	Cancellation reason when available.
<code>carrier_delay,</code> <code>weather_delay,</code> <code>nas_delay,</code> <code>security_delay,</code> <code>late_aircraft_delay</code>	Delay-cause fields, used to infer the dominant cause of a delay.

The selected attributes are enough to compute both required analyses. The airline statistics require carrier, airport, arrival delay, cancellation and date information. The monthly delay report additionally requires departure delay and the different delay-cause fields.

3 Data Preparation

A preparation phase was implemented before running the analyses. This step was necessary because the raw CSV file is not an ideal format for repeated Big Data experiments. CSV files are text-based, require parsing at read time and do not preserve an efficient typed columnar structure. Since the same dataset had to be processed several times by Spark Core, Spark SQL and Hive, I converted the input into Parquet after cleaning and selecting the required fields.

The preparation phase was implemented in `src/data_preparation.py`. Its purpose was simple: read the raw CSV file once, keep only the useful attributes, normalize the values used by the analyses and write the result in Parquet format. The complete script is available in the repository, but the relevant design decisions are summarized below:

- The raw CSV file was read with header information.
- Only the columns required by the two implemented analyses were selected.
- Delay-related null values were replaced with zero to avoid null propagation in aggregations.
- The cancellation flag was represented numerically, so that cancellation rates could be computed using sums and counts.
- The cleaned dataset was written in Parquet format and reused by all technologies.

The most important reason for using Parquet was to avoid repeating expensive CSV parsing for every experiment. Parquet is columnar, so Spark and Hive can read only the columns needed by each job. This is useful in this project because the analyses are mostly aggregation-based and do not require all the attributes of the original dataset.

Some fields were not materialized during preparation but derived later during the analyses. For example, the month was extracted from `fl_date`, the delay range was derived from `dep_delay`, and the dominant cause was inferred from cancellation and delay-cause fields. This kept the prepared dataset general and allowed each technology to implement the derived logic in its own way.

For the second analysis, flights were classified into three departure-delay categories:

Table 2: Departure-delay ranges used in the project

Delay range	Definition
Low delay	Departure delay lower than 15 minutes.
Medium delay	Departure delay between 15 and 60 minutes.
High delay	Departure delay greater than 60 minutes.

The delay or cancellation cause was normalized into a single label. For cancelled flights, the value was derived from the cancellation code. For delayed flights, the dominant cause was selected as the delay component with the largest positive value among carrier, weather, NAS, security and late-aircraft delay. This was necessary because the original dataset stores delay causes in different numerical columns, while the analysis requires counting the most frequent causes per airport, month and delay range.

3.1 Schema issue between Spark, Parquet and Hive

One of the practical problems found during development appeared when Hive tried to read the Parquet dataset generated by Spark. The first Hive table definition did not match the physical schema stored in the Parquet files for the `fl_date` field. As a consequence, Hive failed during schema inspection before the actual analytical query could be executed.

The error was not related to the aggregation logic itself, but to the way Hive interpreted the date field stored by Spark in Parquet. In the first version, the Hive script handled `fl_date` as if it could be freely converted inside the query. However, Spark had already written the column using a Parquet-compatible date representation. The solution was to align the Hive external table definition with the real Parquet schema and then derive the month from that correctly declared field in the Hive view.

The corrected idea was:

```

1  -- boilerplate omitted
2  fl_date DATE,
3  ...
4  STORED AS PARQUET

```

After this change, Hive was able to inspect the Parquet files and the queries for both analyses executed correctly. This issue was useful because it showed that interoperability between Big Data tools depends not only on using a common storage format, but also on declaring schemas consistently across engines.

Overall, the data preparation phase served three purposes: it reduced the amount of data processed by the jobs, provided a consistent input for all technologies and removed repeated parsing costs from the experimental phase.

4 Implemented Analyses

Two analyses were implemented over the prepared flight dataset. Both of them are based on grouping and aggregation, but they differ in complexity. The first analysis is a direct airline-airport aggregation, while the second introduces temporal grouping, delay classification and cause ranking.

4.1 Analysis 3.1: Airline Statistics

The first analysis computes statistics for each airline and departure airport. The objective is to summarize how each carrier behaves at each airport in terms of volume, delay and cancellations. The grouping key is composed of the airline code and the origin airport.

For each airline-airport pair, the following indicators were computed:

- total number of flights;
- minimum arrival delay;
- maximum arrival delay;
- average arrival delay;
- cancellation rate;
- list of months in which the airline operated from that airport.

The cancellation rate was computed as:

$$\text{Cancellation Rate} = \frac{\text{Cancelled Flights}}{\text{Total Flights}} \quad (1)$$

The average arrival delay was computed as:

$$\text{Average Arrival Delay} = \frac{\sum_{i=1}^n \text{arr_delay}_i}{n} \quad (2)$$

where n is the number of flights in the corresponding airline-airport group.

This analysis is relatively direct from a computational point of view. Each flight contributes to exactly one airline-airport group, and the metrics can be obtained using standard aggregation operations. The only non-scalar output is the list of operating months, which requires collecting the distinct months present in each group.

A compact view of the required output is shown in Table 3.

Table 3: Output fields of Analysis 3.1

Field	Meaning
Airline	Operating carrier code.
Departure airport	Airport from which the flights departed.
Number of flights	Total flight count for the airline-airport pair.
Arrival-delay statistics	Minimum, maximum and average arrival delay.
Cancellation rate	Fraction of cancelled flights in the group.
Operating months	Distinct months in which the airline operated from the airport.

From an analytical perspective, this job provides a compact operational profile of each airline at each airport. It can reveal carriers with unusually high cancellation rates, airports where an airline has worse arrival-delay behaviour, or routes that are only active during specific months.

4.2 Analysis 3.2: Delay Report by Airport and Month

The second analysis produces a monthly delay report for each departure airport. This job is more demanding because it does not only aggregate flights by airport and month, but also classifies each flight into a delay-severity range before computing the final statistics.

The grouping key is composed of:

departure airport, month, delay range

The delay ranges are defined in Table 2 (Section 3).

For each airport-month-delay range group, the job computes:

- number of flights;
- average departure delay;
- average arrival delay;
- most frequent delay or cancellation causes.

The two average delay measures are computed as:

$$\text{Average Departure Delay} = \frac{\sum_{i=1}^n \text{dep_delay}_i}{n} \quad (3)$$

$$\text{Average Arrival Delay} = \frac{\sum_{i=1}^n \text{arr_delay}_i}{n} \quad (4)$$

where n is the number of flights in the corresponding airport-month-delay range group.

The cause computation required an additional normalization step. The dataset contains different fields for delay causes: carrier, weather, NAS, security and late aircraft. Instead of keeping these values as separate columns in the final report, I derived a single dominant cause per relevant flight. For delayed flights, the dominant cause was selected as the delay component with the largest positive value. For cancelled flights, the cause was derived from the cancellation code.

This design made it possible to count causes inside each airport-month-delay range group and extract the most frequent ones. Table 4 summarizes the final output.

Table 4: Output fields of Analysis 3.2

Field	Meaning
Departure airport	Airport from which the flights departed.
Month	Month extracted from the flight date.
Delay range	Low, medium or high departure-delay category.
Number of flights	Number of flights in the group.
Average departure delay	Mean departure delay in minutes.
Average arrival delay	Mean arrival delay in minutes.
Top causes	Most frequent delay or cancellation causes in the group.

Compared with Analysis 3.1, this analysis creates more intermediate work. It requires conditional classification, cause normalization and a second aggregation over causes. This is why it is useful for comparing the technologies: Spark SQL and Hive can express most of this logic declaratively, while Spark Core requires more explicit management of the intermediate aggregation state.

5 Technologies and Implementation

The two analyses were implemented using three technologies: Spark SQL, Spark Core and Hive. The goal was to solve the same analytical problems with different programming models and then compare not only their execution time, but also the implementation effort required by each one.

Spark SQL and Hive allow the analyses to be expressed in a declarative way, close to relational algebra. Spark Core, on the other hand, requires explicit construction of keys, values and aggregation functions. This difference was especially visible in Analysis 3.2, where the logic includes delay classification and cause-frequency aggregation.

5.1 Spark SQL Implementation

The Spark SQL implementation was based on DataFrames. The prepared Parquet dataset was loaded as a DataFrame and registered as a temporary view. From that point, the analyses were expressed using SQL queries.

This approach was appropriate because both analyses are naturally structured as grouped aggregations. Analysis 3.1 groups by airline and departure airport, while Analysis 3.2 groups by airport, month and delay category.

Omitting configuration and boilerplate code, which is available in the repository, the core logic of Analysis 3.1 in Spark SQL was solved as follows:

```
1 SELECT
2     op_unique_carrier AS Airline_Code ,
3     origin AS Departure_Airport ,
4     COUNT(*) AS Total_Flights ,
5     MIN(arr_delay) AS Min_Arrival_Delay ,
6     MAX(arr_delay) AS Max_Arrival_Delay ,
7     ROUND(AVG(arr_delay), 2) AS Avg_Arrival_Delay ,
8     ROUND((SUM(cancelled) / COUNT(*)) * 100, 2) AS Cancellation_Rate_Pct ,
9     array_sort(collect_set(month(CAST(fl_date AS TIMESTAMP)))) AS
        Operating_Months
10 FROM flights
11 GROUP BY op_unique_carrier, origin
12 ORDER BY Airline_Code, Departure_Airport
```

This fragment shows why Spark SQL was the most compact implementation. The query directly expresses the expected output: count of flights, delay statistics, cancellation rate and months of operation. The programmer defines the result, while Spark decides the physical execution plan.

For Analysis 3.2, the Spark SQL implementation followed the same idea but added two derived concepts: the delay range and the dominant cause. The delay range was created using conditional expressions, while the cause was derived from cancellation codes or from the largest positive delay component.

The main advantage of this implementation was readability. It was easy to compare the code with the project specification, and the resulting implementation required less manual aggregation logic than Spark Core.

5.2 Spark Core Implementation

Spark Core was implemented using RDDs. This version required a more explicit design because each record had to be transformed into a key-value pair and then combined through distributed aggregation operations.

For Analysis 3.1, the key was the airline-airport pair. The value contained the partial information needed to compute the final statistics: count, arrival-delay values, cancellation count and months of operation. For Analysis 3.2, the key became more complex because it included the departure airport, the month and the delay category.

Omitting configuration and boilerplate code, which is available in the repository, the core of the Spark Core implementation for the delay-range aggregation was:

```
1 def map_stats(row):
2     airport = row.origin if row.origin else "UNKNOWN"
3     month = safe_int(row.month_num)
4
5     dep_delay = safe_float(row.dep_delay)
6     arr_delay = safe_float(row.arr_delay)
7
8     delay_category = categorize_delay(dep_delay)
9
10    key = (airport, month, delay_category)
11    value = (
12        1,
13        dep_delay,
14        arr_delay,
15    )
16
17    return key, value
18
19
20 def reduce_stats(v1, v2):
21     return (
22         v1[0] + v2[0],
23         v1[1] + v2[1],
24         v1[2] + v2[2],
25     )
```

This fragment is the core of the RDD approach. Each flight is mapped to a group, and the reducer combines partial counts and delay sums. The averages are computed only after the aggregation, using the final sums and counts.

The cause computation in Analysis 3.2 required additional logic. For each record, the implementation checked whether the flight was cancelled. If it was cancelled, the cause was derived from the cancellation code. Otherwise, the cause was selected from the delay component with the largest positive value. This logic made the Spark Core solution more verbose than the Spark SQL version.

The main advantage of Spark Core was control. It made the distributed computation explicit and showed clearly how partial results are combined. However, this control came at the cost of more code, more manual state management and higher execution times in the experiments.

5.3 Hive Implementation

Hive was used as a warehouse-style implementation over the same prepared Parquet dataset. The dataset was exposed through an external table, and a view was created to normalize values and derive attributes such as month, route and cause.

Omitting configuration and boilerplate code, which is available in the repository, the central part of the Hive table definition was:

```

1 CREATE EXTERNAL TABLE flights_clean_parquet (
2     fl_date            DATE,
3     op_unique_carrier  STRING,
4     origin             STRING,
5     dest              STRING,
6     dep_delay         DOUBLE,
7     arr_delay         DOUBLE,
8     cancelled         INT,
9     cancellation_code  STRING,
10    carrier_delay      DOUBLE,
11    weather_delay      DOUBLE,
12    nas_delay          DOUBLE,
13    security_delay     DOUBLE,
14    late_aircraft_delay DOUBLE
15 )
16 STORED AS PARQUET
17 LOCATION '${hiveconf:processed_path}';

```

The external table allowed Hive to query the same data prepared by Spark. On top of this table, a view was created to standardize the carrier and airport codes, compute the month and derive the delay or cancellation cause.

For Analysis 3.2, Hive used common table expressions to separate the computation into stages: first computing delay statistics, then counting causes, then ranking them and finally joining the results. The central idea is shown below:

```

1 WITH stats AS (
2     SELECT
3         origin AS Departure_Airport,
4         month AS Month,
5         CASE
6             WHEN COALESCE(dep_delay, 0) < 15 THEN '1_Low (<15m)'
7             WHEN COALESCE(dep_delay, 0) <= 60 THEN '2_Medium (15-60m)'
8             ELSE '3_High (>60m)'
9         END AS Delay_Category,
10        COUNT(*) AS Total_Flights,
11        ROUND(AVG(COALESCE(dep_delay, 0)), 2) AS Avg_Dep_Delay,
12        ROUND(AVG(COALESCE(arr_delay, 0)), 2) AS Avg_Arr_Delay
13    FROM flights_clean
14    GROUP BY origin, month,
15        CASE
16            WHEN COALESCE(dep_delay, 0) < 15 THEN '1_Low (<15m)'
17            WHEN COALESCE(dep_delay, 0) <= 60 THEN '2_Medium (15-60m)'
18            ELSE '3_High (>60m)'
19        END
20 ),
21 ranked_causes AS (
22     SELECT
23         Departure_Airport,
24         Month,
25         Delay_Category,
26         cause,
27         Cause_Count,
28         ROW_NUMBER() OVER (
29             PARTITION BY Departure_Airport, Month, Delay_Category
30             ORDER BY Cause_Count DESC, cause ASC
31         ) AS rk

```

```

32 FROM cause_counts
33 )

```

This implementation was expressive because the analysis fits well into SQL. The use of common table expressions also made the query easier to structure: each step had a specific purpose. However, Hive required more setup than Spark SQL because the external table had to match the physical Parquet schema generated by Spark.

5.4 Implementation Comparison

Table 5 summarizes the main differences observed during implementation.

Table 5: Comparison of implementation approaches

Aspect	Spark SQL	Spark Core	Hive
Programming model	Declarative	RDD-based	Declarative SQL warehouse
Main abstraction	DataFrame / SQL view	Key-value RDDs	External table and view
Aggregation logic	Managed by Spark SQL	Manually defined	Managed by Hive
Code length	Short	Longer	Medium
Implementation difficulty	Low	High	Medium
Best part	Readable and optimized	Full control	Familiar SQL model
Main drawback	Less low-level control	More manual aggregation state	Schema setup and query overhead

In this project, Spark SQL was the most balanced implementation. It was concise, readable and efficient. Spark Core was the most explicit implementation and helped to understand the real aggregation process, but it required more code and was slower. Hive was useful as a SQL warehouse alternative, especially after solving the Parquet schema compatibility issue.

6 Experimental Setup

The experiments were designed to compare the three implementations under similar conditions. All jobs used the same prepared Parquet dataset, so the comparison focused on the processing model of each technology rather than on differences in input format.

The execution was carried out in two environments:

- a local machine, used as a baseline;
- an AWS EMR cluster, used to test the same workload in a distributed environment.

6.1 Local Environment

The local execution was performed on a personal machine. This environment was useful as a baseline because it avoided the overhead of cluster startup, distributed scheduling and remote storage access.

Table 6: Local execution environment

Component	Specification
Processor	13th Gen Intel(R) Core(TM) i7-13700H
RAM	32 GB
GPU memory	8 GB
Execution mode	Local Spark execution and local Hive execution

The local machine was powerful enough to process the prepared dataset efficiently. This point is relevant when interpreting the results: the local environment was not a weak baseline, so the cluster was not guaranteed to be faster for this dataset size.

6.2 AWS EMR Cluster

The cluster experiments were executed on AWS EMR. The prepared Parquet dataset was stored in Amazon S3 and accessed from the EMR jobs. This setup is closer to a real Big Data architecture, where storage and computation are separated.

Table 7: AWS EMR cluster configuration

Component	Configuration
Platform	AWS EMR
EMR release	emr-6.15.0
Hadoop version	3.3.6
Spark version	3.4.1
Hive version	3.1.3
Primary nodes	1
Core nodes	1
Instance type	m4.large
Storage layer	Amazon S3
Cluster size label	1-primary-1-core

The cluster was intentionally small. The choice of `m4.large` instances and a single core node was mainly constrained by the course laboratory environment and by the need to keep the execution simple and reproducible. This configuration was enough to validate the distributed execution of Spark and Hive, but it was not designed to maximize performance.

No aggressive Spark tuning was applied. The jobs were executed with the default EMR Spark configuration and with the scripts included in the repository. This makes the experiments easier to reproduce, although it also means that the reported cluster times should be interpreted as the performance of a simple educational EMR deployment, not as an optimized production cluster.

6.3 Input Sizes

To evaluate scalability, Spark SQL and Spark Core were executed with three input sizes:

- 25% of the prepared dataset;

- 50% of the prepared dataset;
- 100% of the prepared dataset.

The complete prepared dataset contained 7,079,081 records. The smaller fractions were used to observe how execution time changed as the amount of data increased.

Hive was executed only on the full prepared dataset. This decision was taken because the main objective of the Hive implementation was to compare a warehouse-style SQL approach with the Spark implementations on the complete data.

Table 8: Input sizes used in the experiments

Technology	Analyses	Input sizes
Spark SQL	3.1 and 3.2	25%, 50%, 100%
Spark Core	3.1 and 3.2	25%, 50%, 100%
Hive	3.1 and 3.2	100%

6.4 Measurement Methodology

Execution time was measured at job level. For each technology and analysis, the measured time includes the analytical computation and the writing of the output. The initial data preparation phase was measured separately and was not included in the analysis execution times, because it is shared by all technologies.

For Spark SQL and Spark Core, the benchmark script executed the two analyses for each input fraction and stored the elapsed time in CSV format. For Hive on EMR, the reported values correspond to wall-clock time measured by the execution wrapper. This includes the practical overhead of submitting and completing the Hive queries in the cluster environment.

The consolidated metrics used in the report are stored in:

`results/metrics/execution_times_all.csv`

Representative output samples are stored in:

`results/samples/`

The figures used in the results section were generated from the collected metrics and stored in:

`report/figures/`

This organization separates raw data, processed data, metrics and report artifacts, making the project easier to reproduce and inspect.

7 Results

This section presents representative outputs of the implemented analyses and the execution-time results obtained in the local and EMR environments. The complete result files are available in the repository, while this report includes only selected samples and summarized metrics to keep the discussion focused.

Analysis 3.1 — Output samples

Table 9 shows the first ten rows produced by the Spark SQL implementation of Analysis 3.1 on the full dataset.

Table 9: First 10 rows of Analysis 3.1 — Spark SQL (100% dataset)

Airline	Airport	Flights	Min Arr.	Max Arr.	Avg Arr.	Canc.%	Months
9E	ABE	1017	−47.0	575.0	0.66	1.38	1–12
9E	ABY	38	−32.0	321.0	4.37	0.00	1,2,3,11,12
9E	AEX	878	−34.0	979.0	9.50	1.03	1–12
9E	AGS	1603	−36.0	1091.0	6.82	2.06	1–12
9E	ALB	1089	−51.0	394.0	−4.30	1.93	1–12
9E	ATL	26877	−43.0	1207.0	2.42	1.09	1–12
9E	ATW	176	−34.0	420.0	9.34	0.00	1,4–10
9E	AUS	379	−35.0	1225.0	14.07	0.53	1–7,11
9E	AVL	675	−43.0	949.0	1.87	2.07	1–12
9E	AZO	14	−22.0	71.0	−2.93	0.00	1

The Spark Core implementation produces numerically identical results for Analysis 3.1. Table 10 shows the corresponding output from the RDD-based job, sorted by the same key. The maximum absolute difference between the two implementations across all 1,738 airline-airport pairs is 0.01 minutes for average arrival delay and 0.01 percentage points for cancellation rate, which is explained by floating-point rounding differences in the final computation step. The row counts and all integer fields are bitwise equal.

Table 10: First 10 rows of Analysis 3.1 — Spark Core / RDD (100% dataset)

Airline	Airport	Flights	Min Arr.	Max Arr.	Avg Arr.	Canc.%	Months
9E	ABE	1017	−47.0	575.0	0.66	1.38	1–12
9E	ABY	38	−32.0	321.0	4.37	0.00	1,2,3,11,12
9E	AEX	878	−34.0	979.0	9.50	1.03	1–12
9E	AGS	1603	−36.0	1091.0	6.82	2.06	1–12
9E	ALB	1089	−51.0	394.0	−4.30	1.93	1–12
9E	ATL	26877	−43.0	1207.0	2.42	1.09	1–12
9E	ATW	176	−34.0	420.0	9.34	0.00	1,4–10
9E	AUS	379	−35.0	1225.0	14.07	0.53	1–7,11
9E	AVL	675	−43.0	949.0	1.87	2.07	1–12
9E	AZO	14	−22.0	71.0	−2.93	0.00	1

One minor formatting difference exists between the implementations: the operating-months field. Spark SQL and Spark Core produce a comma-separated list ordered numerically (e.g. 1,2,3,...,12), because the months are collected as integers and sorted before formatting. The Hive implementation, which collects months as strings before applying `collect_set`, produces lexicographic ordering instead (e.g. 1–10–11–12–2–3-...). This difference does not affect the analytical correctness of the result: the same set of months is present in both cases. It is an output-format consequence of using a string-based collection function in HQL versus an integer-sorted array in Spark.

Analysis 3.2 — Output samples

Table 11 shows the first ten rows of Analysis 3.2 produced by Spark SQL. Table 12 shows the corresponding output from Spark Core. As with Analysis 3.1, the numerical values are identical

across both implementations; the only format difference is in the cause separator used internally (Spark SQL uses a ranked struct expression while Spark Core uses a sorted Python list), but the final string representation in both outputs is the same.

Table 11: First 10 rows of Analysis 3.2 — Spark SQL (100% dataset)

Airport	Mo.	Range	Flights	Avg Dep.	Avg Arr.	Top Causes
ABE	1	Low (<15m)	288	−5.30	−13.90	CANCEL_WEATHER:11
ABE	1	Med. (15–60m)	31	34.32	31.45	LATE_AIRCRAFT:12; CARRIER:8; NAS:3
ABE	1	High (>60m)	30	241.30	234.03	LATE_AIRCRAFT:14; CARRIER:12; NAS:2
ABE	2	Low (<15m)	300	−6.58	−19.31	CANCEL_WEATHER:3
ABE	2	Med. (15–60m)	20	29.70	9.60	CARRIER:5; LATE_AIRCRAFT:3
ABE	2	High (>60m)	14	227.93	217.93	CARRIER:6; LATE_AIRCRAFT:6; NAS:1
ABE	3	Low (<15m)	341	−6.15	−18.26	CANCEL_NAS:1; CAN- CEL_WEATHER:1
ABE	3	Med. (15–60m)	28	30.68	19.89	LATE_AIRCRAFT:7; CARRIER:4; NAS:4
ABE	3	High (>60m)	23	173.43	163.39	CARRIER:9; LATE_AIRCRAFT:8; NAS:6
ABE	4	Low (<15m)	306	−6.44	−17.13	CANCEL_NAS:1; CAN- CEL_WEATHER:1

Table 12: First 10 rows of Analysis 3.2 — Spark Core / RDD (100% dataset)

Airport	Mo.	Range	Flights	Avg Dep.	Avg Arr.	Top Causes
ABE	1	Low (<15m)	288	-5.30	-13.90	CANCEL_WEATHER:11
ABE	1	Med. (15–60m)	31	34.32	31.45	LATE_AIRCRAFT:12; CARRIER:8; NAS:3
ABE	1	High (>60m)	30	241.30	234.03	LATE_AIRCRAFT:14; CARRIER:12; NAS:2
ABE	2	Low (<15m)	300	-6.58	-19.31	CANCEL_WEATHER:3
ABE	2	Med. (15–60m)	20	29.70	9.60	CARRIER:5; LATE_AIRCRAFT:3
ABE	2	High (>60m)	14	227.93	217.93	CARRIER:6; LATE_AIRCRAFT:6; NAS:1
ABE	3	Low (<15m)	341	-6.15	-18.26	CANCEL_NAS:1; CAN- CEL_WEATHER:1
ABE	3	Med. (15–60m)	28	30.68	19.89	LATE_AIRCRAFT:7; CARRIER:4; NAS:4
ABE	3	High (>60m)	23	173.43	163.39	CARRIER:9; LATE_AIRCRAFT:8; NAS:6
ABE	4	Low (<15m)	306	-6.44	-17.13	CANCEL_NAS:1; CAN- CEL_WEATHER:1

This sample shows why the delay-range classification is useful. At airport ABE, low-delay flights in January had an average departure delay of -5.30 minutes, while high-delay flights in the same month had an average departure delay of 241.30 minutes. A single average value for the airport would hide this difference.

7.1 Execution Time on the Full Dataset

Table 13 compares the execution times obtained on the full prepared dataset. These values are the main reference for comparing Spark SQL, Spark Core and Hive under the same input size.

Table 13: Execution time on the full dataset

Environment	Technology	Analysis	Cluster Size	Time (s)
Local	Spark SQL	3.1	local	3.28
Local	Spark SQL	3.2	local	5.46
Local	Spark Core	3.1	local	44.70
Local	Spark Core	3.2	local	91.65
Local	Hive	3.1	local	11.27
Local	Hive	3.2	local	25.85
EMR	Spark SQL	3.1	1 primary, 1 core	10.63
EMR	Spark SQL	3.2	1 primary, 1 core	6.52
EMR	Spark Core	3.1	1 primary, 1 core	108.00
EMR	Spark Core	3.2	1 primary, 1 core	205.97
EMR	Hive	3.1	1 primary, 1 core	152.00
EMR	Hive	3.2	1 primary, 1 core	153.00

Spark SQL was the fastest implementation on the full dataset. Spark Core was significantly slower, especially for Analysis 3.2, where the implementation had to manage more manual aggregation logic. Hive performed reasonably in the local environment, but its wall-clock time on EMR was much higher because the measurement included query submission and cluster execution overhead.

7.2 Scalability Results

Spark SQL and Spark Core were also evaluated using 25%, 50% and 100% of the prepared dataset. Tables 14 and 15 report the results for both analyses.

Table 14: Scalability results for Analysis 3.1

Environment	Technology	Input Size	Cluster Size	Time (s)
Local	Spark SQL	25%	local	3.44
Local	Spark SQL	50%	local	1.93
Local	Spark SQL	100%	local	3.28
Local	Spark Core	25%	local	17.39
Local	Spark Core	50%	local	27.22
Local	Spark Core	100%	local	44.70
EMR	Spark SQL	25%	1 primary, 1 core	11.36
EMR	Spark SQL	50%	1 primary, 1 core	7.18
EMR	Spark SQL	100%	1 primary, 1 core	10.63
EMR	Spark Core	25%	1 primary, 1 core	33.18
EMR	Spark Core	50%	1 primary, 1 core	56.83
EMR	Spark Core	100%	1 primary, 1 core	108.00

Table 15: Scalability results for Analysis 3.2

Environment	Technology	Input Size	Cluster Size	Time (s)
Local	Spark SQL	25%	local	5.15
Local	Spark SQL	50%	local	3.58
Local	Spark SQL	100%	local	5.46
Local	Spark Core	25%	local	43.90
Local	Spark Core	50%	local	58.68
Local	Spark Core	100%	local	91.65
EMR	Spark SQL	25%	1 primary, 1 core	12.05
EMR	Spark SQL	50%	1 primary, 1 core	6.18
EMR	Spark SQL	100%	1 primary, 1 core	6.52
EMR	Spark Core	25%	1 primary, 1 core	55.54
EMR	Spark Core	50%	1 primary, 1 core	103.97
EMR	Spark Core	100%	1 primary, 1 core	205.97

The scalability results show a clear difference between the two Spark implementations. Spark Core execution time grows more visibly as the input size increases, especially in Analysis 3.2. Spark SQL remains more stable because its execution engine can optimize structured transformations, column access and aggregation planning.

Some Spark SQL results are not strictly monotonic. This is normal in this type of experiment. With these input sizes, fixed costs such as job initialization, scheduling, partitioning and filesystem access can affect the final time as much as the amount of data processed.

7.3 Charts

The execution-time comparison is also represented graphically. Figure 1 compares the three technologies on the complete dataset in the local environment. Since both environments produced the same ranking between technologies, the local results are used here because they isolate the processing time of each implementation from the cluster-startup and S3 access overhead visible in Table 13.

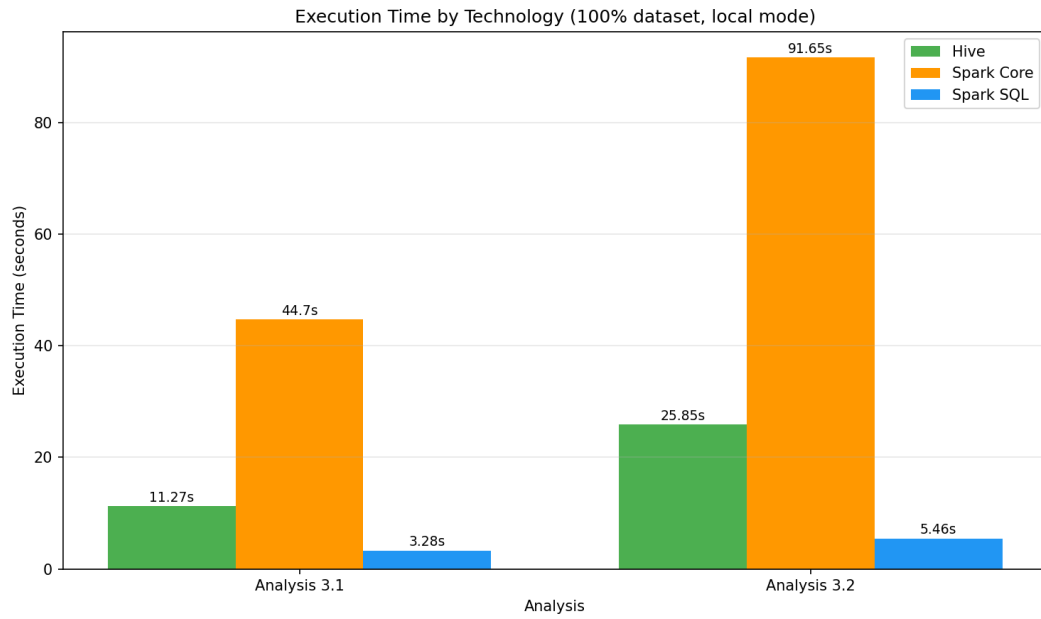


Figure 1: Execution time comparison by technology

Figure 2 shows the scalability behaviour for Analysis 3.1.

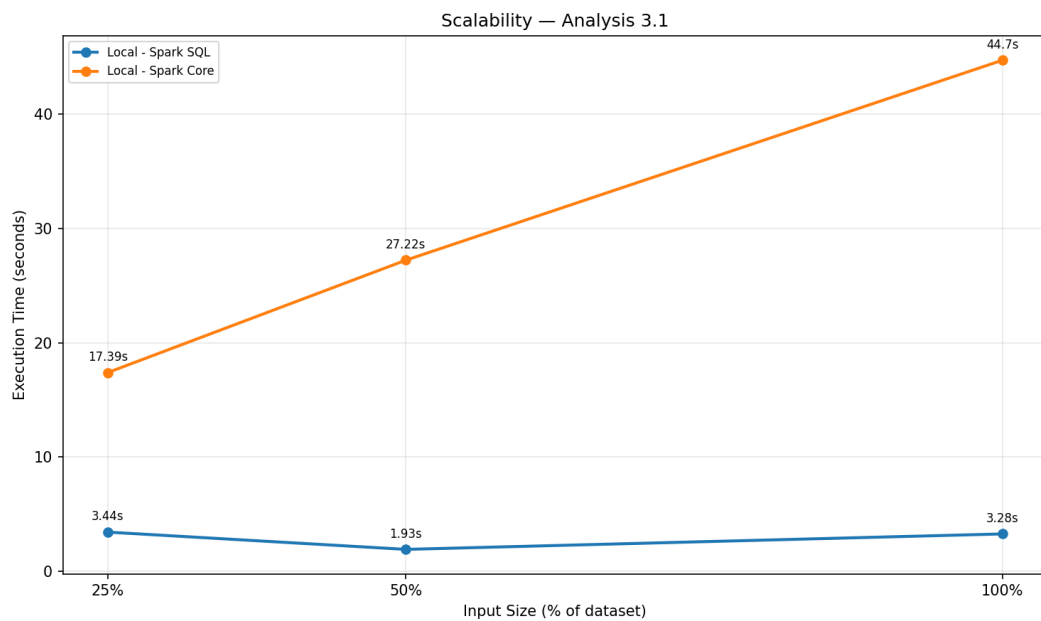


Figure 2: Scalability analysis for Analysis 3.1

Figure 3 shows the scalability behaviour for Analysis 3.2.

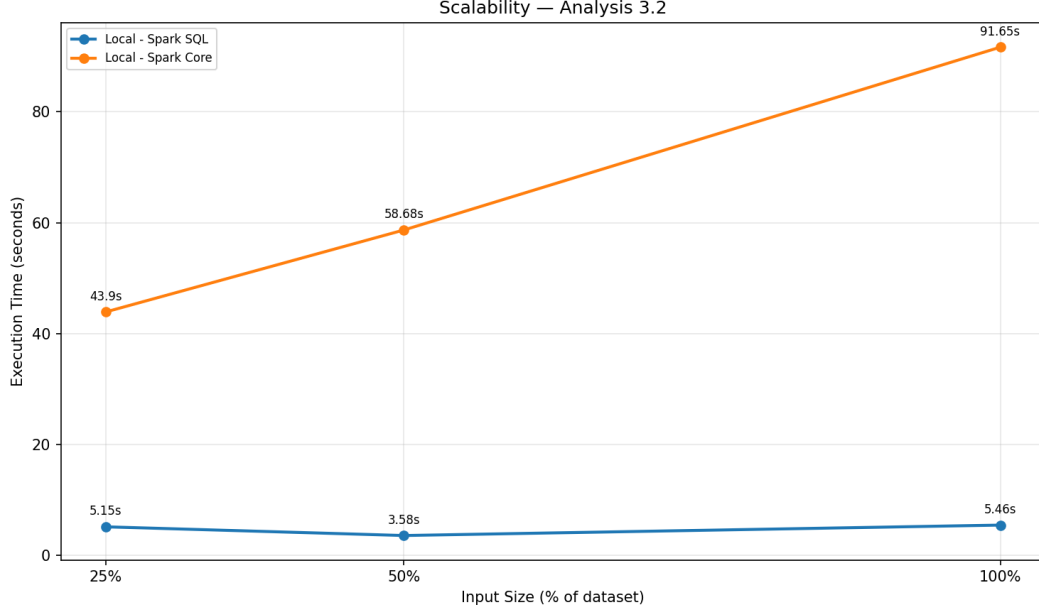


Figure 3: Scalability analysis for Analysis 3.2

The figures confirm the same trend observed in the tables: Spark SQL was consistently the best-performing option, while Spark Core became more expensive as the workload became more complex. Hive was useful for the SQL warehouse comparison, but its cluster execution time was affected by the overhead of the small EMR setup.

8 Discussion

The experiments show that the three technologies were able to produce the required outputs, but they did not behave in the same way. The differences were visible both in the implementation phase and in the execution-time results.

8.1 Expressiveness and Development Effort

Before discussing expressiveness and development effort, it is worth noting that all three implementations produced consistent analytical results. For Analysis 3.1, the Spark SQL and Spark Core outputs contain exactly the same 1,738 airline-airport pairs, with a maximum numerical difference of 0.01 minutes in average arrival delay, attributable to floating-point rounding in the final division step. For Analysis 3.2, the 11,902 group rows produced by both Spark implementations are numerically identical for flight counts, and differ by at most 0.01 minutes in the average delay fields. The Hive output was verified against these values and produces the same numerical statistics.

One format difference is worth noting. The operating-months field in Analysis 3.1 is sorted numerically in both Spark implementations (1,2,3,...,12), because months are collected as integers before joining. The Hive implementation uses `collect_set` on string-typed values, which causes lexicographic ordering of the resulting set (1-10-11-12-2-3-...). This is a known behaviour of HQL's `collect_set` function and does not indicate an error in the query logic; the same twelve months are present in both outputs. It is a concrete example of how different tools, even when solving the same analytical problem over the same data, can produce outputs with small format differences depending on their internal type handling.

Spark SQL was the most convenient technology for this project. Both analyses are based on structured operations: selecting columns, deriving new attributes, grouping records and computing aggregations. These operations fit naturally into the DataFrame and SQL model. As a result, the Spark SQL implementation was close to the statement of the problem.

Spark Core required more work. In this implementation, I had to explicitly decide the key of each aggregation, the value stored for each record and how two partial results were merged. This was manageable for Analysis 3.1, but became more verbose in Analysis 3.2 because the job also needed delay categories and cause-frequency information.

Hive was also expressive because the analyses are relational. Once the external table and the analytical view were defined, most of the work could be written as HQL queries. However, Hive required more setup than Spark SQL. In particular, the external table had to be consistent with the Parquet schema generated by Spark, which introduced an additional source of errors.

Table 16 summarizes the development experience.

Table 16: Development comparison

Criterion	Spark SQL	Spark Core	Hive
Code complexity	Low	High	Medium
Readability	High	Medium	High
Manual aggregation logic	Low	High	Low
Setup effort	Low	Medium	Medium–High
Debugging difficulty	Low	Medium–High	Medium–High

The main conclusion from the implementation phase is that a higher-level API was enough for this workload. Spark Core gave more control, but the extra control did not translate into better performance or simpler code for these specific analyses.

8.2 Efficiency

Spark SQL was the fastest technology in the experiments. On the full dataset, it executed Analysis 3.1 in 3.28 seconds locally and 10.63 seconds on EMR. For Analysis 3.2, it required 5.46 seconds locally and 6.52 seconds on EMR. These results indicate that Spark SQL handled the structured aggregation workload efficiently.

Spark Core was slower in all comparable experiments. On the full dataset, the local execution time was 44.70 seconds for Analysis 3.1 and 91.65 seconds for Analysis 3.2. On EMR, the times increased to 108.00 seconds and 205.97 seconds. The difference was especially large in Analysis 3.2 because the RDD implementation had to manage more intermediate state manually.

Hive had acceptable local times but much higher wall-clock times on EMR. The cluster execution required 152 seconds for Analysis 3.1 and 153 seconds for Analysis 3.2. The cluster figures reflect the overhead of query submission and scheduling in a small EMR environment rather than a property of Hive itself: the local execution, where Hive ran against a pre-loaded dataset without distributed coordination, was considerably faster.

8.3 Scalability and Cluster Overhead

The scalability results show a clear difference between Spark SQL and Spark Core. Spark Core execution time increased more consistently with input size. This was expected because the RDD

implementation performs more manual aggregation and produces more intermediate processing, especially in Analysis 3.2.

Spark SQL did not always show monotonic growth when moving from 25% to 50% and 100% of the dataset. This happened because, at this scale, fixed costs still matter. Job initialization, scheduling, partitioning, JVM startup effects and file access can influence the final time. With larger datasets, the data-processing cost would probably dominate these fixed costs more clearly.

One of the most important lessons of the project is that using a cluster does not automatically mean obtaining faster execution. The local machine used in the experiment had strong hardware, while the EMR cluster was intentionally small: one primary node and one core node. In this configuration, the overhead of distributed execution was significant.

The EMR setup was useful to validate the cluster execution and to test the integration with S3, but it was not large enough to outperform the local machine consistently. In practice, moving to a cluster is justified when the dataset is too large for a single machine, when the workload needs to be repeated at scale, or when multiple jobs must run in parallel. For this project size, the local machine was still competitive.

8.4 Impact of Shuffle and Aggregation

Both analyses require grouping records by categorical keys. This implies shuffle, because records with the same key must be brought together before the final aggregation can be computed.

Analysis 3.1 is less expensive because it groups by airline and departure airport. The aggregation state is simple: counts, delay sums, minimum and maximum values, cancellation counts and operating months.

Analysis 3.2 is heavier. It groups by airport, month and delay range, and it also computes the most frequent causes. This creates more intermediate information and makes the aggregation less trivial. The effect is visible in the execution times: Analysis 3.2 was slower than Analysis 3.1, especially in Spark Core.

Spark SQL and Hive hide most of the shuffle details behind their execution engines. Spark Core exposes them more directly. This is useful for understanding how the computation works, but it also makes the implementation easier to write inefficiently.

8.5 Impact of Data Preparation

The data preparation phase was important for both performance and reproducibility. Working directly with the raw CSV file would have forced every job to parse text repeatedly. By converting the dataset to Parquet, the analyses could operate on a typed, columnar and more compact representation.

This was especially useful because the analyses only needed a subset of the original attributes. Parquet allowed Spark and Hive to read the relevant columns more efficiently. It also helped to make the comparison fairer, because all technologies used the same prepared input.

However, the Parquet conversion also introduced the schema compatibility issue with Hive. This was the main technical problem found during the project. Spark wrote the data correctly, but Hive required the external table definition to match the Parquet schema. The issue was solved by correcting the declaration of the date field and deriving the month from the correctly interpreted column. This reinforced the idea that storage format compatibility is as important as the analytical logic itself.

8.6 Overall Evaluation

Table 17 summarises the final comparison across the three dimensions evaluated in this section.

Table 17: Final comparison of the technologies

Criterion	Spark SQL	Spark Core	Hive
Best strength	Fast structured analytics	Low-level control	SQL warehouse model
Main weakness	Less manual control	Verbose aggregation logic	Setup and cluster overhead
Performance	Best overall	Slowest Spark option	Good locally, slower on EMR
Implementation effort	Low	High	Medium
Best fit	This project workload	Custom distributed logic	Batch SQL analysis

The results confirm that no single technology is optimal in all respects. Spark SQL was the best fit for this specific workload because the analyses were structured, aggregation-oriented and well-suited to the DataFrame model. Spark Core was slower and more verbose for these jobs, but its explicit key-value model is still relevant when the computation requires fine-grained control that SQL cannot easily express. Hive added value as a warehouse-style reference point, and its overhead on the small EMR cluster should not be read as a fundamental limitation of the technology.

9 Reproducibility

The complete source code of the project is available in the following GitHub repository:

https://github.com/JuanMiguelPinos/flight_delay.git

The repository contains the source code for data preparation, the three implementations, the execution scripts, the experimental metrics, representative output samples and the documentation required to reproduce the project.

The raw dataset is not included in the repository because of its size. To reproduce the project, the original CSV file must be downloaded separately and placed in the input directory described in the repository documentation. After that, the preparation script can be executed to generate the cleaned Parquet dataset used by Spark Core, Spark SQL and Hive.

Table 18: Main repository structure

Path	Purpose
<code>src/</code>	Source code for data preparation, Spark Core, Spark SQL and Hive analyses.
<code>scripts/</code>	Scripts for local and cluster execution.
<code>docs/</code>	Additional documentation, including the AWS EMR execution guide.
<code>results/metrics/</code>	Execution-time measurements used in the comparison.
<code>results/samples/</code>	Representative samples of the produced analytical outputs.
<code>report/figures/</code>	Charts used in the final report.
<code>tools/</code>	Helper scripts used to consolidate metrics and extract samples.
<code>README.md</code>	Main instructions for understanding and reproducing the project.

The local workflow can be summarized in three steps. First, the raw CSV file is cleaned and converted to Parquet. Second, the analytical jobs are executed with Spark Core, Spark SQL and Hive. Third, the resulting metrics and sample outputs are collected for the report.

The cluster workflow follows the same logical structure, but uses Amazon S3 as storage and AWS EMR as the execution environment. The repository includes a dedicated guide describing the cluster configuration, the organization of the S3 bucket and the execution of the cluster script.

The consolidated metrics used in this report are stored in:

`results/metrics/execution_times_all.csv`

The representative outputs shown in the results section are stored in:

`results/samples/`

This organization keeps the repository lightweight while still preserving the elements needed to inspect and reproduce the work. Large files, raw data and intermediate generated outputs are intentionally excluded.

10 Conclusion

This project implemented and compared Spark Core, Spark SQL and Hive on the 2024 Flight Delay Dataset. The original CSV dataset was first prepared by selecting the relevant attributes, normalizing delay-related values and converting the result into Parquet format. Two analyses were then implemented: airline statistics by departure airport and monthly delay reports by airport and delay range.

The results show that Spark SQL was the best option for this workload. The reason is not only that it was faster, but also that the analyses fit naturally into its programming model. Both jobs are based on structured operations such as grouping, aggregation, conditional expressions and ranking. Spark SQL allowed these operations to be expressed clearly while relying on the Spark optimizer for the physical execution.

Spark Core produced correct results, but required more manual work. In the RDD implementation, the grouping keys, partial values and reduction logic had to be explicitly designed. This made the implementation more verbose, especially for the second analysis, where delay categories and cause-frequency information had to be managed manually. The execution times confirmed this additional cost: Spark Core was consistently slower than Spark SQL.

Hive was useful as a warehouse-style alternative. It allowed the prepared Parquet dataset to be queried using external tables and HQL. However, Hive also introduced one of the most relevant technical issues of the project: the external table schema had to be aligned with the Parquet schema generated by Spark. Once the date-field mismatch was corrected, Hive was able to execute the analyses correctly. This issue highlighted the importance of schema consistency when several Big Data tools share the same storage format.

The comparison between local and EMR execution was also important. The local machine used in the experiments was relatively powerful, while the EMR cluster was intentionally small, with one primary node and one core node. For this dataset size, the overhead of the cluster was significant. Job submission, distributed scheduling, communication and remote storage access sometimes made the cloud execution slower than the local execution.

This does not mean that distributed processing is unnecessary. Rather, it shows that scaling out only becomes beneficial when the data volume, workload complexity or concurrency requirements justify the overhead. In this project, the EMR cluster was valuable to validate the distributed execution and to test the integration with S3, but it was not large enough to outperform the local environment consistently.

Overall, the main lesson is that the best technology depends on the workload. For structured analytical jobs over prepared columnar data, Spark SQL provided the best balance between performance, readability and development effort. Spark Core was useful to understand the lower-level mechanics of distributed aggregation, but was less convenient for this specific task. Hive remained a valid option for SQL-oriented batch analysis, especially when the data is already organized in a compatible format such as Parquet.